Exp-2:

Design Zero-shot, One-shot, and Few-shot prompts to generate a 200-word blog on"Applications of Artificial Intelligence in Healthcare

Aim:

To design specialized, task-specific prompts for three different functional applications —text summarization, professional email creation, and creative content generation —and to evaluatethe quality of the responses generated by an LLM for each function.

Algorithm :

1. Start.
2. Import the required libraries and configure the API key for the LLM.
3. Define a prompt template for the summarization function that specifies the desired summary length and tone.
4. Define a prompt template for the email generation function that specifies the recipient, context, and tone (formal/informal).
5. Define a prompt template for the content generation function that specifies the topic, target audience, and writing style.
6. Pass the corresponding input text/context to each prompt template.
7. Send each completed prompt to the LLM API and receive the generated response.
8. Display the summary, email, and generated content separately.
9. Evaluate each output for coherence, relevance, and completeness with respect to the task.
10. Stop.

Python Program:

```
import openai

openai.api_key =
"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"

def call_llm(prompt): response = openai.ChatCompletion.create( model="gpt-3.5-
turbo",

messages=[{"role": "user", "content": prompt}], temperature=0.5

)

return response["choices"][0]["message"]["content"].strip()

Summarization function

def summarize(text):

prompt = f"""Summarize the following passage in 3 concise sentences,

keeping the tone neutral and factual:
```

{text} return call_llm(prompt)

def generate_email(context, recipient, tone="formal"):

prompt = f"""Write a {tone} email to {recipient} regarding the

following context. Include a subject line, greeting, body, and closing signature.

Context: {context} return call_llm(prompt)

Content generation function

def generate_content(topic, audience, style="informative"):

prompt = f"""Write a short {style} blog paragraph on the topic "{topic}" for an audience of {audience}. Keep it engaging and under 150 words. return call_llm(prompt)

article = "Artificial Intelligence is rapidly transforming industries by automating tasks, enabling data driven decisions, and creating new products and services across healthcare, finance, and education."

print("SUMMARY:\n", summarize(article)) print(":\n", generate_email(

"requesting a project deadline extension of one week",

"Project Manager")) print("\nCONTENT:\n", generate_content(

"benefits of renewable energy", "college students"))

Input:

Article text on Artificial Intelligence (for summarization); context "requesting a project deadline extension of one week" ; topic "benefits of renewable energy" for audience "college students" (for content generation).

Output

SUMMARY:

AI is transforming industries by automating tasks and enabling data-driven decisions. It is being adopted across healthcare, finance, and education. New products and services are emerging as a result of this transformation.

Result

Three functional prompts were successfully designed and executed for summarization, email creation, and content generation. Each prompt template, by clearly specifying the task, format, tone, and constraints, produced outputs that were coherent, contextually relevant, and appropriate to the intended function, confirming that task-specific prompt design significantly improves output quality compared to generic prompting.